HexaLogic: A Browser-Native AT89C51 and ARM Educational Emulator with Instruction-Level Debugging and Experimental Virtual Hardware

Ashwinder Pal Singh HexaLogic Project
Browser-Based Embedded Systems Laboratory

Abstract

This paper presents HexaLogic, a browser-native embedded-systems emulator designed around a custom AT89C51/8051 simulation core, a functional teaching-oriented ARM execution mode, an assembler pipeline, live debugging views, and an experimental virtual-hardware layer. The platform was built to provide a Keil-like workflow in the browser: source editing, assembly listing, run/step controls, register and flag inspection, RAM/ROM views, breakpoints, reverse stepping, clock-aware timing, and interactive I/O observation. The 8051 side models the MCS-51 programming surface used in typical AT89C51 laboratory exercises, including accumulator operations, register banks, direct and indirect RAM addressing, bit-addressable ports, SFR-backed I/O, timers, interrupts, serial state, and machine-cycle timing. The ARM mode is intentionally narrower: it is a functional educational sandbox that supports a practical A32-style subset, Keil-like source directives, data labels, endian control, GPIO-style memory-mapped I/O, timers, interrupts, and multiply instructions including `UMLAL`. The current virtual-hardware subsystem supports LEDs, LED arrays, seven-segment displays, switches, and steppers as digital teaching models; it is useful for demonstrating firmware-to-I/O behavior, but full electrical co-simulation remains future work. Validation combines unit tests, integration tests, API workflow tests, frontend contract tests, and a regression-audit script. The latest local validation reported 192 passing pytest cases, a successful production frontend build, and 10 passing hard-audit checks.

Keywords: AT89C51, 8051, ARM, emulator, assembler, virtual laboratory, embedded systems education, debugging, machine cycles, virtual hardware

1 Introduction

The 8051 family remains an important teaching architecture because it exposes the details that many modern high-level environments hide: accumulator-centered computation, register-bank selection, bit-addressable memory, SFR-controlled ports, explicit stack behavior, timer overflow, interrupt entry, and software-visible machine cycles [1, 3]. The AT89C51 variant keeps that instructional value while providing a concrete device target with 4 KB Flash, 128 bytes of internal RAM, 32 programmable I/O lines, two 16-bit timers, six interrupt sources, and an MCS-51-compatible instruction set and pinout [2].

Traditional embedded tools such as Keil μVision combine source editing, project management, program debugging, and simulation in a single desktop environment [5]. That integration is powerful, but it also creates access friction in classrooms where learners may use different operating systems, locked-down machines, or low-power devices. Browser-based laboratories address this access problem, and prior work on virtual microcontroller labs has shown that simulated laboratories

can reduce equipment constraints while preserving meaningful engineering practice [6, 7]. HexaLogic follows the same broad educational direction, but focuses on instruction-level transparency and a browser IDE workflow rather than on photorealistic laboratory equipment.

HexaLogic was built as a custom emulator platform. Its purpose is not to wrap an external 8051 simulator, and it does not claim to be a full silicon-accurate verifier. The design goal is more practical: provide a professional, reproducible, browser-accessible environment where a student can paste assembly, assemble it, connect a virtual LED or bus device, run the code, and inspect the exact CPU and I/O state that follows from the program.

2 Design Goals

The project was guided by six concrete goals.

1. **Source-to-state visibility.** Every assemble/run/step action should expose enough state for the learner to understand what changed and why.
2. **AT89C51-first behavior.** The 8051 model should prioritize the real programming model used in AT89C51 lab code: ports, SFRs, register banks, timers, interrupts, and machine-cycle timing.
3. **Keil-like workflow without desktop lock-in.** The interface should feel like a compact embedded debugger: editor, assembler output, target explorer, RAM/ROM views, register watch, trace, and console.
4. **Multi-architecture extensibility.** The runtime should allow more than one target architecture without duplicating the whole application stack.
5. **Clock-aware execution.** Timing shown by hardware and metrics should derive from configured clock frequency and architecture-specific cycle units.
6. **Truthful virtual hardware.** The hardware view should show digital behavior that is actually modeled, while leaving analog/electrical accuracy for future work.

3 System Architecture

HexaLogic uses a browser frontend and a Python backend. The frontend is built with HTML, CSS, JavaScript, the Monaco editor, and Vite production bundling. Monaco is a browser-based source editor derived from the Visual Studio Code editor stack [10]; Vite provides the static production build path [11]; the deployed frontend can be served from a static host whose build command and publish directory are controlled through Netlify-style configuration [12]. The backend uses Flask as the WSGI application layer; Flask documentation describes the WSGI deployment model used to run HTTP applications behind a WSGI server [9].

The application boundary is JSON over HTTP. The backend owns simulation state through isolated sessions, while the frontend renders snapshots and deltas. The important backend layers are:

- `SimulatorSession`, which binds architecture, assembler, CPU, virtual hardware, run mode, clock, and session export/import.

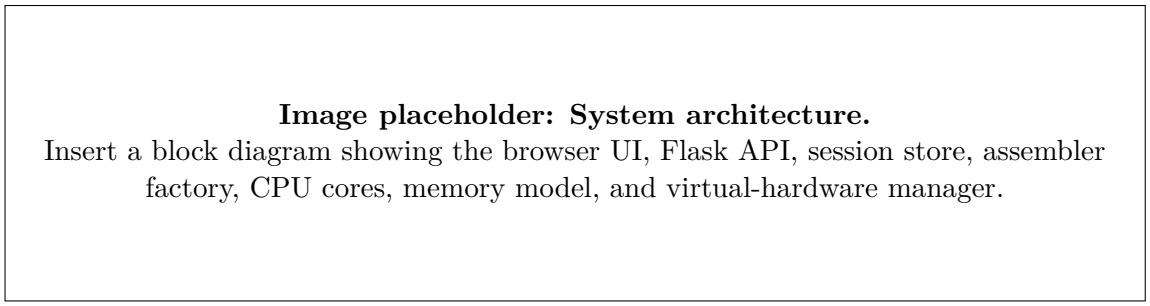

Figure 1: HexaLogic system architecture. Recommended image: a clean layered block diagram from browser controls to backend execution and hardware synchronization.

Table 1: Major Runtime Components

Component	Responsibility
Session store	Isolates user state and supports import/export payloads.
Assembler factory	Selects 8051 or ARM assembler based on architecture.
CPU factory	Selects architecture-specific execution core.
Memory model	Represents ROM, IRAM, SFR, XRAM, and endian-aware word access.
Debugger model	Tracks trace entries, breakpoints, call stack, and reverse deltas.
Hardware manager	Evaluates virtual devices from pins, buses, MMIO, and signal logs.
Frontend renderer	Draws editor state, memory tables, debugger panes, hardware canvas, and waveform views.

- `Assembler8051` and `AssemblerARM`, which turn source into program images, listings, labels, ROM bytes, and data initializers.
- `CPU8051` and `CPUARM`, which execute instruction streams and expose runtime snapshots.
- `VirtualHardwareManager`, which converts CPU pin/MMIO snapshots into device states, signal logs, timing metrics, and validation warnings.
- Flask API endpoints under `/api/v2`, which expose assembly, run control, reset, clock setting, architecture switching, memory editing, session import/export, and virtual-hardware operations.

4 AT89C51/8051 Model

The AT89C51 model is the primary target. It is implemented as a PC-driven MCS-51-compatible educational core rather than as a cycle-by-cycle transistor or bus simulator. The model supports the main state surfaces students expect when working with AT89C51 assembly:

- accumulator `A`, `B`, `DPTR`, `SP`, `PC`, `PSW`, and register banks `R0`–`R7`;
- internal RAM, code ROM, XRAM sample space, and SFR-backed I/O;

- direct, indirect, immediate, register, bit, and relative addressing forms;
- branch/call/return behavior, stack updates, and source-line trace mapping;
- port SFRs `P0`–`P3`, including latch and pin-level distinction for hardware synchronization;
- timers, interrupt flags, interrupt vectors, priority ordering, and serial status state where implemented;
- machine-cycle accounting, with the default 8051 timing basis of oscillator clock divided by 12.

The machine-cycle decision matters. Many 8051 devices execute one classic machine cycle per 12 oscillator periods, and the MCS-51 documentation treats machine cycles as the timing unit for instruction execution [1]. HexaLogic therefore reports `clock_hz` separately from effective 8051 execution frequency. At 12 MHz, one 8051 machine cycle is modeled as 1 microsecond. A delay loop can therefore be tested by counting machine cycles and converting through

$$t_{8051} = \frac{\text{machine cycles}}{\text{clock\_hz}/12}.$$

The hardware timing layer uses that same effective frequency so LED warnings, stable-time metrics, and signal timestamps are not detached from CPU execution.

4.1 Example: User-Pasted Port Program

The following program is a representative AT89C51 workflow:

```
ORG 0000H
MOV A,#01H
MOV R0,#20H
MOV @R0,A
INC A
MOV P1,A
SJMP $
END
```

The program writes `01H` through indirect RAM address `20H`, increments `A` to `02H`, writes `02H` to `P1`, and loops forever. Therefore, after `MOV P1,A`, bit `P1.0` is 0 and `P1.1` is 1. A virtual LED connected to `P1.0` is off in an active-high model, while a LED connected to `P1.1` is on. This exact workflow is now covered by regression testing, including device persistence across run execution.

4.2 Example: Delay Loop Timing

The LED-array rotating program below exercises a common classroom delay structure:

```
ORG 0000H
MOV A,#01H
START:
MOV P1,A
ACALL DELAY
```

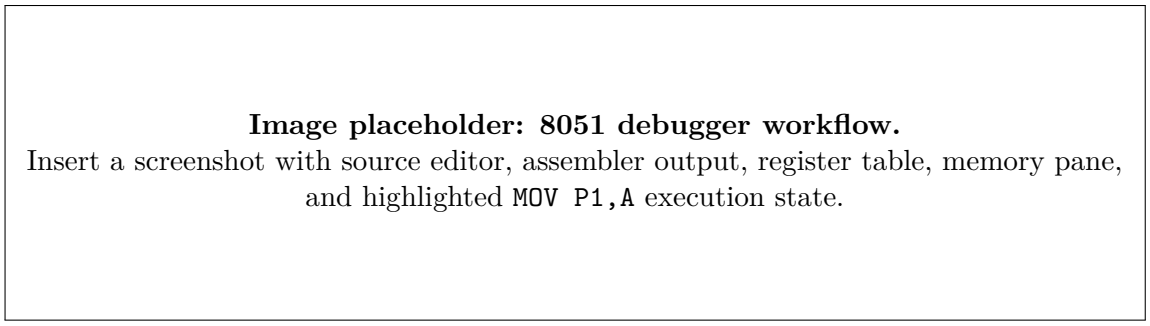

Figure 2: AT89C51 source-to-state workflow. Recommended image: a screenshot after assembling the pasted `P1` program with register `A=02H` and `P1` visible.

```
RL A
SJMP START
DELAY:
MOV R7,#200
D1: MOV R6,#250
D2: DJNZ R6,D2
DJNZ R7,D1
RET
END
```

In the current model, the first two `MOV P1,A` executions at a 12 MHz clock occur at cycle counts 2 and 100,611, giving a period of 100,609 machine cycles or 0.100609 seconds. That behavior is validated as a unit test. This is not a one-second delay; it is about one tenth of a second at 12 MHz. HexaLogic exposes the clock input so the learner can distinguish code timing from oscillator choice instead of guessing from the UI.

5 ARM Educational Sandbox

The ARM mode is intentionally described as an educational sandbox, not a complete ARM Architecture Reference Manual implementation. The ARM architecture manuals define a very broad execution environment [4]; HexaLogic implements a practical subset that is useful for teaching assembly dataflow and memory-mapped control. The supported surface includes:

- 16 general registers `R0`–`R15`, aliases `SP`, `LR`, and `PC`;
- condition flags `N`, `Z`, `C`, and `V`;
- little-endian and big-endian word access;
- data-processing instructions such as `MOV`, `MVN`, `ADD`, `ADDS`, `ADC`, `SUB`, `SBC`, `AND`, `ORR`, `EOR`, `CMP`, and `TST`;
- shifts and conditional execution suffixes for the implemented instruction families;
- `LDR`, `STR`, `B`, `BL`, `BX`, and pseudo stack operations `PUSH`/`POP`;
- multiply family support: `MUL`, `MLA`, `UMULL`, `UMLAL`, `SMULL`, and `SMLAL`;

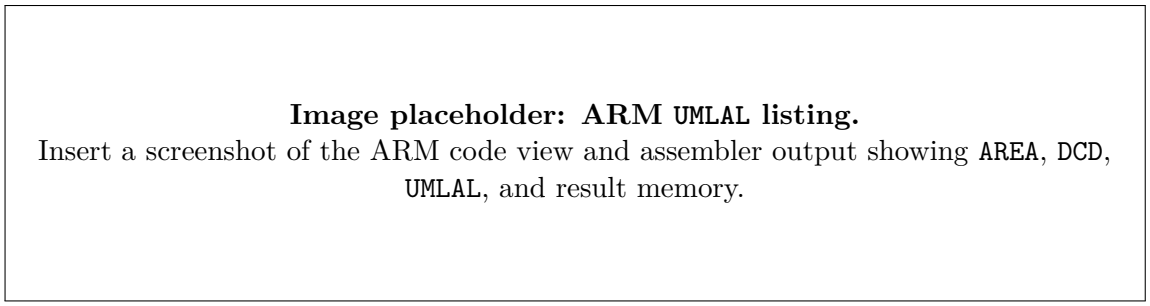

Figure 3: ARM educational sandbox workflow. Recommended image: the Keil-style ARM example after run, showing the UMLAL listing and result words.

- Keil-like source conveniences: AREA, ENTRY, labels without colons, DCD, and literal-address loading such as LDR R0, =NUM1;
- a small GPIO/timer/IRQ MMIO model for virtual hardware demonstrations.

The following Keil-style sequence is representative:

```
AREA PROGRAM, CODE, READONLY
ENTRY
MAIN
LDR R0, =NUM1
LDR R1, =NUM2
LDR R2, =RESULT
LDR R3, [R0]
LDR R4, [R0, #4]
LDR R5, [R1]
LDR R6, [R1, #4]
UMLAL R3, R4, R5, R6
STR R3, [R2]
STR R4, [R2, #4]
...
AREA PROGRAM, DATA, READONLY
NUM1 DCD 0xFFFFFFFF
     DCD 0x00000001
NUM2 DCD 0x00000001
     DCD 0x00000002
RESULT DCD 0x00000000
       DCD 0x00000000
END
```

The current assembler places data labels in XRAM beginning at the configured data base, initializes words from DCD, emits executable code for supported instructions, and allows the CPU to run the program. Regression tests verify that the example produces the expected result words and reaches R8=4 in the multiword-add path.

6 Debugger and User Interface

The frontend is arranged as an IDE-like workspace. The left side presents target, execution state, registers, flags, timers, and call stack. The center presents the source editor, assembler listing, debugger console, and trace timeline. The right side presents RAM, XRAM, code ROM, and runtime metrics. The hardware view replaces the center with a board, component palette, MCU pin model, device inspector, port monitor, component state, validation output, signal log, and logic analyzer.

The interface intentionally removes user-resizable panel behavior. Earlier prototypes allowed panes to grow and shrink during interaction, which made the debugger hard to read when a console filled with repeated loop messages. The current UI fixes the panel structure and makes overflow areas scrollable. The debugger console also compacts repeated run-loop messages so tight loops do not expand the workspace indefinitely.

The toolbar includes assemble, run, step into, step over, step out, step back, pause, stop, run-to-cursor, reset, architecture selection, endian selection, debug mode, execution mode, speed, clock frequency, base conversion, import/export, waveform view, help, dark mode, breakpoint clearing, and memory edit. The clock control exposes preset frequencies such as 11.0592 MHz and 12 MHz as well as a custom integer Hz input, making timing assumptions visible.

7 Experimental Virtual Hardware

The virtual-hardware system is implemented as a digital state model. It is useful for showing whether firmware drives a pin high or low, whether a bus value changes, whether a stepper pattern appears, and whether a signal is toggling too fast to be human-visible. The implemented devices are:

- single LED connected to one pin;
- LED array connected to an 8-bit bus;
- seven-segment display connected to an 8-bit segment bus;
- switch connected to one input pin;
- stepper motor connected to a 4-bit bus.

For 8051, the signal catalog exposes `P0.0`–`P3.7` and bus groups such as `P1`. The model treats `P0` as open-drain metadata and distinguishes latch and pin state for port reads. For ARM, the catalog exposes `GPIOA.0`–`GPIOA.15` and word-oriented MMIO state.

This hardware layer is deliberately labeled experimental. It does not yet model LED current, pull-up resistor strength, capacitive loading, analog transitions, bus contention with real electrical equations, oscillator tolerance, external crystal startup, or full peripheral waveforms. It is a digital teaching model designed to make firmware-visible state understandable. Full mixed-signal simulation is future work.

8 Validation

HexaLogic is validated through layered tests rather than by visual inspection alone.

The latest local validation used:

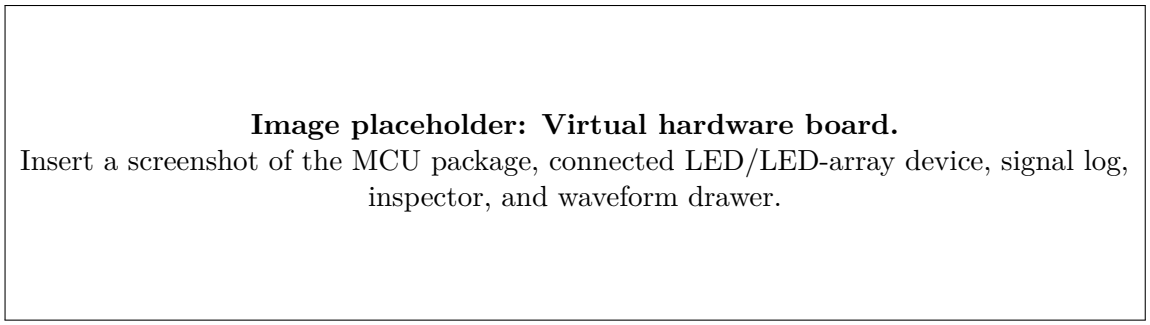

Figure 4: Experimental virtual-hardware board. Recommended image: a connected `P1` LED or LED-array workflow with visible pin state and signal history.

Table 2: Current Validation Coverage

Validation Layer	Examples of Checked Behavior
8051 core tests	interrupts, timers, port behavior, stack/call flow, cycle counts, delay loops, bit operations.
ARM core tests	flags, shifts, endian memory, `UMLAL`, signed/unsigned multiply-long, Keil-style directives.
API tests	session isolation, assembly, run/step/reset, clock setting, hardware persistence, CORS session headers.
Frontend tests	required UI IDs, handler bindings, non-growing console behavior, static panel layout contracts.
Hardware tests	LED/switch propagation, signal logs, timing unit metadata, warning generation, ARM GPIO output.
Audit script	end-to-end trace contracts, reverse step, hardware device persistence, waveform proxy, example matrix.

- `python -m pytest -q`: 192 passed tests;
- `npm run build`: successful Vite production build;
- `python scripts/regression_audit.py --strict`: 10 passing audit checks, zero warnings, zero failures.

The validation suite includes the two user-relevant workflows that motivated the latest hardening pass. First, the pasted 8051 program that writes `P1=02H` is assembled, connected to virtual LEDs, and run; the LED devices persist and pin states match the program. Second, the ARM Keil-style example with `UMLAL` and `DCD` data labels is assembled and executed with expected result words.

9 Limitations

The most important limitation is scope. The AT89C51 model is a serious educational emulator, but it is not a certified hardware replacement. It models the programming behavior needed for many assembly labs, not every analog or board-level effect. The ARM mode is narrower still: it is a functional teaching subset with approximate instruction timing, not a full ARMv7 architectural compliance model.

The browser frontend also favors observability over full project management. The target explorer is intentionally lightweight and does not yet implement a multi-file project system. Watchpoints exist in the backend API, but a dedicated watchpoint UI has not yet been added. Serial RX injection exists in the backend, while a full serial terminal panel remains future work.

10 Future Scope

Future development should focus on five areas:

1. **Electrical virtual hardware.** Extend the current digital model with resistor, current, loading, pull-up, and bus-contention calculations.
2. **Peripheral depth.** Add richer UART terminal visualization, interrupt timeline views, timer waveform capture, and SFR-specific inspectors.
3. **ARM expansion.** Broaden instruction coverage while keeping the UI honest about which architecture profile and timing model are implemented.
4. **Project workflow.** Add multi-file source organization, user-named examples, and reproducible lab templates.
5. **Educational evaluation.** Run classroom studies comparing debugging speed, state comprehension, and lab completion quality against desktop-only workflows and other 8051 simulators.

11 Conclusion

HexaLogic demonstrates that a browser-based emulator can provide a professional embedded debugging experience without hiding the low-level behavior that makes microcontroller education valuable. The AT89C51/8051 core models the essential programming surface of classroom assembly work, including ports, SFRs, registers, memory, interrupts, timers, and machine-cycle timing. The ARM sandbox adds a practical second architecture with Keil-style source support and multiply-long instructions such as `UMLAL`. The virtual hardware layer already supports useful digital demonstrations, while its partial nature is explicit and reserved for future expansion into richer electrical simulation.

The strongest contribution is the integration: source, assembly listing, execution controls, memory/register state, hardware pins, clock-aware timing, and regression-tested workflows are presented in one browser environment. That combination makes HexaLogic suitable for instruction, practice, and iterative debugging of embedded assembly programs.

References

[1] Intel Corporation, *MCS-51 Microcontroller Family User's Manual*, Order No. 272383-002, Feb. 1994. [Online]. Available: https://www.bitsavers.org/components/intel/8051/MCS-51_Users_Manual_Feb94.pdf

[2] Atmel Corporation, *AT89C51: 8-bit Microcontroller with 4K Bytes Flash*, Data Sheet, Rev. 0265G–02/00. [Online]. Available: https://www.keil.com/dd/docs/datashts/atmel/at89c51_ds.pdf

[3] M. A. Mazidi, J. G. Mazidi, and R. D. McKinlay, *The 8051 Microcontroller and Embedded Systems: Using Assembly and C*, 2nd ed. Upper Saddle River, NJ, USA: Pearson/Prentice Hall, 2006.

[4] Arm Ltd., *Arm Architecture Reference Manual Armv7-A and Armv7-R edition*, DDI 0406. [Online]. Available: https://documentation-service.arm.com/static/5f8daeb7f86e16515cdb8c4e

[5] Arm Keil, "μVision IDE overview." [Online]. Available: https://www.keil.com/products/uvision/

[6] J. J. Richardson and N. Adamo-Villani, "A virtual embedded microcontroller laboratory for undergraduate education: Development and evaluation," *Engineering Design Graphics Journal*, vol. 74, no. 3, 2010, doi: https://doi.org/10.18260/edgj.v74i3.223.

[7] K. Choi, S. Han, S. Kim, D. Kim, J. Lim, D. Ahn, and C. Jeon, "A combined virtual and remote laboratory for microcontroller," in *Hybrid Learning and Education*, Lecture Notes in Computer Science, vol. 5685. Berlin, Germany: Springer, 2009, pp. 66–76.

[8] J. Rogers, "EdSim51DI: Educational simulator for the 8051." [Online]. Available: https://edsim51.com/

[9] Pallets, "Deploying to production – Flask Documentation." [Online]. Available: https://flask.palletsprojects.com/en/stable/deploying/

[10] Microsoft, "Monaco Editor." [Online]. Available: https://github.com/microsoft/monaco-editor

[11] Vite, "Building for production." [Online]. Available: https://vite.dev/guide/build.html

[12] Netlify, "Build configuration overview." [Online]. Available: https://docs.netlify.com/build/configure-builds/overview/